Change-Impact Analysis & Combinatorial Testing

Slides by Sam Allen

Change Detection Analysis

- Change Detection Analysis seeks to find how changes to individual model constants affect model outputs.
- It does this by changing a single model constant across its range of values while holding all other model constants at their default values.
- Provides a simple form of qualitative analysis (one-constant-at-a-time) but misses interactions among constants and their joint effect on output changes

Assumptions

- Initial analysis was built on a few key assumptions.
- First, that not all the constants had an effect on the model outputs.
- Second, model constants did not influence what values other model constants could take.
- Third, model constants’ effects on model outputs aren’t determined by preconditions being met.
- These assumptions underlie the initial results from the change detection analysis plugin.

Usefulness

- By discussing with the domain scientists and examining `LakeTemperature` source code, these assumptions turned out to be wrong, meaning the results don't include all of model outputs that the model constants affect.
- However, this doesn't discount the results of Change Detection Analysis.
- It led to the discovery that certain constants only effect model output under certain conditions.
- Its results are also still valid and useful when specifically referring to the default values used in the test cases.

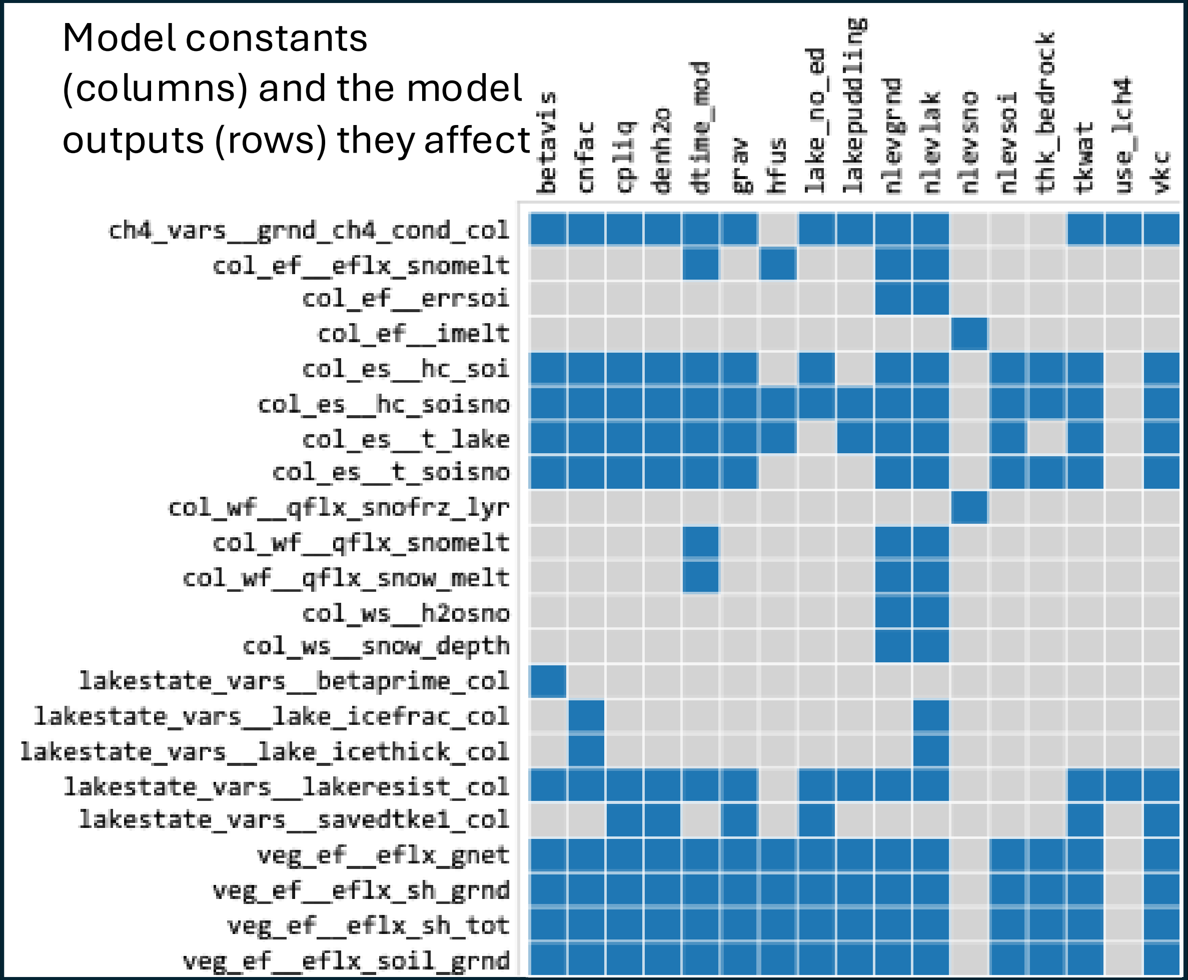
Rationale

- To determine which model constants affect which model outputs, 11 cases were created for every model constant, where all model constants were held at their default values except the one which was being tested, which was linearly interpolated to get 11 values evenly spread across its valid range.
- This assumed that each model constant's values are independent of each other and that no preconditions have to be met for a constant to have an effect on model outputs.

Preliminary Results

Model constants that had no effect on any model output

```
cpice
denice
depthcrit
iulog
mixfact
pudz
tkair
tkice
```



Model outputs that didn't change in response to any model constants

```
top_pp__active
lun_pp__topounit
lun_pp__itype
lun_pp__lakpoi
lun_pp__urbpoi
lun_pp__active
lakestate_vars__etal_col
lakestate_vars__lake_raw_col
lakestate_vars__ks_col
lakestate_vars__ws_col
soilstate_vars__watsat_col
soilstate_vars__tkmg_col
soilstate_vars__tkdry_col
soilstate_vars__tksatu_col
soilstate_vars__csol_col
solarabs_vars__sabg_patch
solarabs_vars__sabg_lyr_patch
solarabs_vars__fsds_nir_d_patch
solarabs_vars__fsds_nir_i_patch
solarabs_vars__fsr_nir_d_patch
solarabs_vars__fsr_nir_i_patch
col_pp__gridcell
col_pp__topounit
col_pp__landunit
col_pp__itype
col_pp__active
col_pp__snl
col_pp__dz
col_pp__z
col_pp__zi
col_pp__dz_lake
col_pp__z_lake
col_pp__lakedepth
col_es__t_grnd
col_ws__h2osoi_liq
col_ws__h2osoi_ice
col_ws__frac_iceold
col_wf__qflx_snofrz
veg_pp__topounit
veg_pp__landunit
veg_pp__column
veg_pp__itype
veg_pp__active
```

Problems/Solutions

- So far, the combinatorial test cases were only using the constants assumed to have an impact on model outputs. Now, they include all of the constants.
- The Change Detection analysis might be redone in the future by examining preconditions through LakeTemperature source code but future combinatorial testing and change-impact analysis will not depend on these results.

Change-Impact Analysis Overview

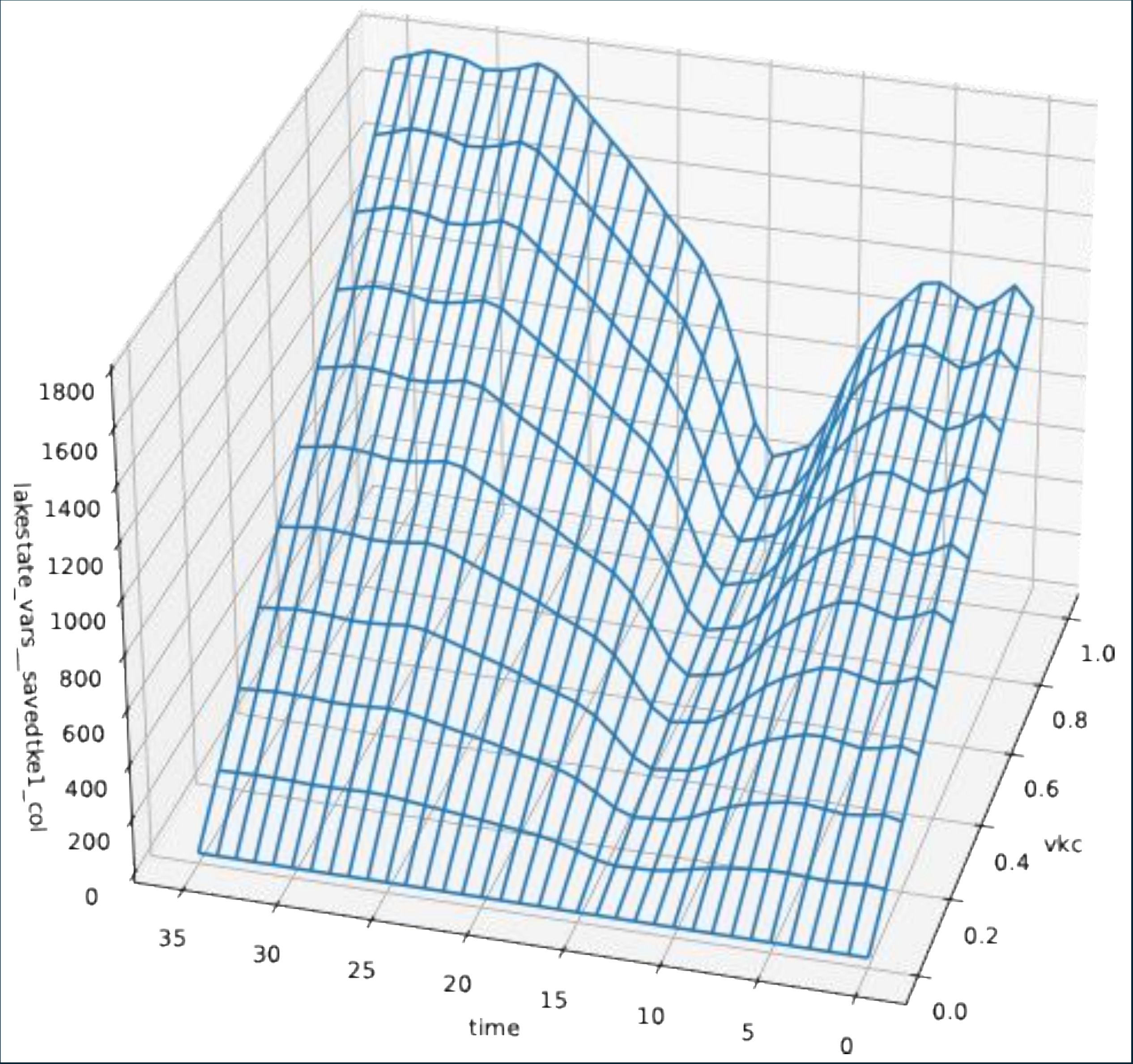
- The goal is to visualize the impact of model constants on model outputs.
- Some challenges are:
  - Most of the 25 model constants we change affect multiple model outputs.
  - The 75 model outputs are multidimensional. The column dimension goes from 0 to 344.
  - Contrary to an initial assumption, some model constants only affect model output when certain preconditions are met.
- To solve this, we create a 3D graph using time as the x-axis, the one model constant we changed as the y-axis, and a single column of a single model output as the z-axis.

Graphing Process

- Output data in the form of a NetCDF file is used.
- From the output data we pulled the values for the model constant *vkc*, which represents the Von Kármán Constant.
- Time is interpolated from 0 to 35.
- From the output data we pulled the values for the model output *lakestate_vars__savedtke1_col*–the eddy conductivity at the top lake level saved for the next step–at the column numbered 336.
- Using matplotlib, all three dimensions were graphed.

An Example Graph

- This graph shows how the model constant *vkc* affects the model output *lakestate_vars__savedtke1_col* over a period of 35 time steps for the lake column numbered 336.



Why Combinatorial Testing (CT)?

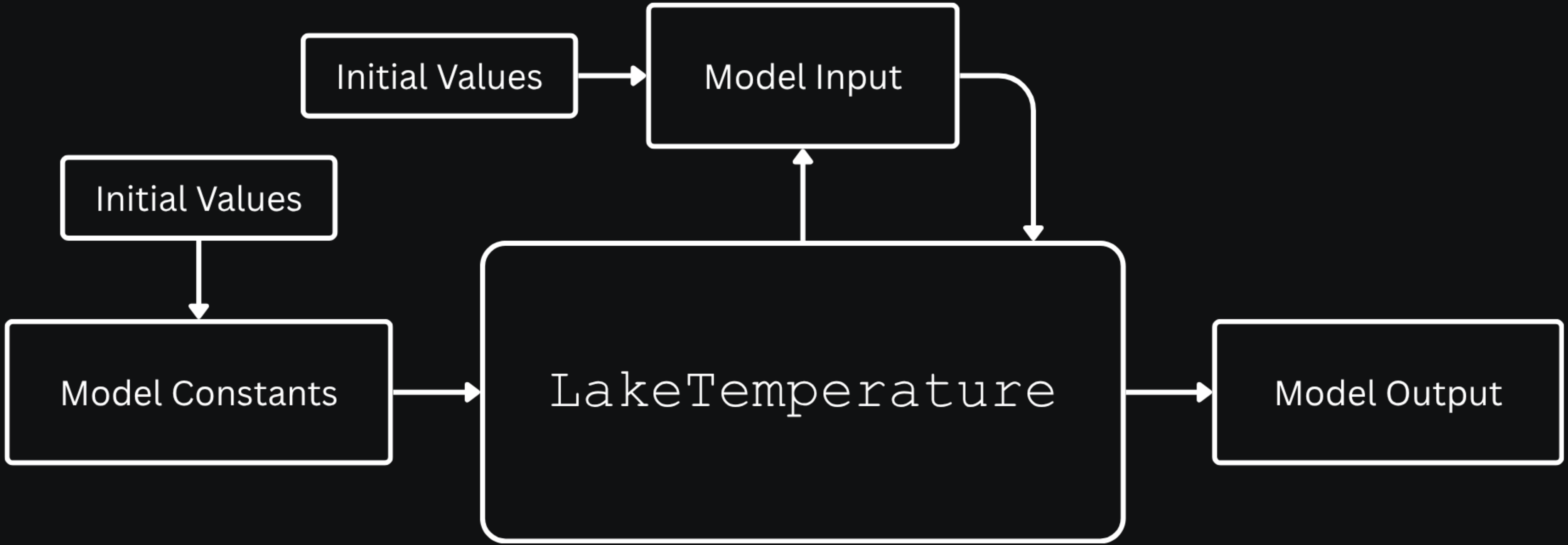
- Ideally, `LakeTemperature` should be validated against all possible combinations of model constant values to ensure that no combination results in erroneous output.
- Exhaustive testing is neither practical nor feasible due to the vast number of all possible combinations.
- Combinatorial testing substantially reduces the number of required test cases while achieving near-exhaustive coverage.
- Assuming most of the bugs are triggered by the interactions of model constants

Background of Combinatorial Testing

- NIST research found most software bugs are caused by interactions of only a small number of parameters[1].
- By testing all combinations of a certain number of parameters, most software bugs can be found with much fewer test cases.
- NIST's ACTS tool can generate covering arrays from 1-way to 6-way or with mixed coverage[2].

1. *Automated Combinatorial Testing for Software (ACTS)*. NIST Computer Security Division. (2016, July 20). https://csrc.nist.rip/groups/SNS/acts/
2. *User Guide for ACTS*. NIST. (n.d.). https://csrc.nist.gov/csrc/media/Projects/automated-combinatorial-testing-for-software/documents/acts_user_guide_for_basic1.0_and_advanced3.3_versions_nov23.pdf

Visualization of Model Constants, Inputs, and Output



Application of CT to `LakeTemperature`

- The model constants, which we can control when running `LakeTemperature`, are used as the parameters for the combinatorial test cases.
- 3 constants are Booleans, 7 are integers, and 15 are floats.
- Booleans can be either 0 or 1.
- For integers, we linearly interpolate over their ranges to get up to 11 values.
- For floats, we linearly interpolate over their ranges to get exactly 11 values.

NetCDF Excerpt for Illustration

bool lakepuddling = 0, 1 ;

Int nlevsno = 0, 1, 2, 3, 4, 5 ;

Int depthcrit = 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35 ;

Float thk_bedrock = 2, 2.2, 2.4, 2.6, 2.8, 3, 3.2, 3.4, 3.6, 3.8, 4 ;

Test Oracle

- Scientific software verification is challenging due to often lacking a test oracle.
- We implemented checks that validate model outputs, supported by the Fault Finding Analysis Plugin.
- Checks include ranges of outputs, signs, enumerated values, finiteness, relationships to other outputs, and certain physical laws and constraints.

Sign Checks Overview

- Check that a certain model output is positive
- 10 out of 46 checks are of this type

Example Check

```python
def check_water_snow_equivilant_not_negative(
    test_col_ws_h2osno: npt.NDArray
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_ws_h2osno):
        return CheckStatus.SKIPPED
    test_col_ws_h2osno = NonFiniteValuesHandler.mask_non_finite_values(
        test_col_ws_h2osno)

    assert_not_negative(test_col_ws_h2osno, "col_ws%ws_h2osno")
```

```python
def assert_not_negative(variable: npt.NDArray, name: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all(variable >= 0.0), f"Negative values found in {name}"
```

All Sign Checks

File Name	Check Name
ch4_conductance_check.py	check_methane_conductance_not_negative
physical_bounds_check.py	check_water_snow_equivalent_not_negative
physical_bounds_check.py	check_snow_depth_not_negative
physical_bounds_check.py	check_snow_freeze_rate_not_negative
physical_bounds_check.py	check_snow_melt_flux_not_negative
physical_bounds_check.py	check_net_snow_melt_not_negative
physical_bounds_check.py	check_snow_melt_heat_flux_not_negative
physical_bounds_check.py	check_lake_water_transport_resistance_not_negative
physical_bounds_check.py	check_saved_eddy_conductivity_not_negative
physical_bounds_check.py	check_soil_heat_content_not_negative

Range Checks Overview

- Check that a certain model output is a fraction
- 2 out of 46 checks are of this type

Example Check

```
def check_surface_absorption_fraction_is_fraction(
    test_lakestate_vars_betaprime_col: npt.NDArray
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_lakestate_vars_betaprime_col):
        return CheckStatus.SKIPPED
    test_lakestate_vars_betaprime_col = NonFiniteValuesHandler.mask_non_finite_values(
        test_lakestate_vars_betaprime_col)

    assert_is_frac(test_lakestate_vars_betaprime_col, "lakestate_vars%betaprime_col")
```

```
def assert_is_frac(variable: npt.NDArray, check_combinme: str) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(variable):
        return CheckStatus.SKIPPED
    variable = NonFiniteValuesHandler.mask_non_finite_values(variable)
    assert np.all((variable >= 0.0) & (variable <= 1.0)), (
        f"Non-fractional values found in {name}"
    )
```

All Range Checks

File Name	Check Name
physical_bounds_check.py	check_lake_layer_ice_fraction_is_fraction
physical_bounds_check.py	check_surface_absorption_fraction_is_fraction

Finiteness Checks Overview

- Check that a certain model output is finite
- 22 out of 46 checks are of this type

Example Check

```python
def check_lake_temperature_finite(test_col_es_t_lake: npt.NDArray) -> None:
    assert_variable_is_finite(test_col_es_t_lake, "col_es%t_lake")
```

```python
def assert_variable_is_finite(variable: npt.NDArray, name: str) -> None:
    assert np.all(np.isfinite(variable)), f"Infinite or NaN values found in {name}"
```

All Finiteness Checks (Part 1)

File Name	Check Name
finite_values_check.py	check_lake_temperature_finite
finite_values_check.py	check_soil_snow_temperature_finite
finite_values_check.py	check_ground_net_heat_flux_finite
finite_values_check.py	check_ground_sensible_heat_flux_finite
finite_values_check.py	check_total_sensible_heat_flux_finite
finite_values_check.py	check_ground_heat_flux_finite
finite_values_check.py	check_snow_melt_heat_flux_finite
finite_values_check.py	check_snow_freeze_rate_finite
finite_values_check.py	check_snow_melt_water_flux_finite
finite_values_check.py	check_new_snow_melt_rate_finite
finite_values_check.py	check_snow_water_equivalent_finite

All Finiteness Checks (Part 2)

File Name	Check Name
finite_values_check.py	check_snow_depth_finite
finite_values_check.py	check_lake_transport_resistance_finite
finite_values_check.py	check_saved_eddy_conductivity_finite
finite_values_check.py	check_ground_methane_conductance_finite
finite_values_check.py	check_energy_conservation_residual_finite
finite_values_check.py	check_snow_layer_flag_finite
finite_values_check.py	check_soil_heat_content_finite
finite_values_check.py	check_combined_heat_content_finite
finite_values_check.py	check_surface_absorption_finite
finite_values_check.py	check_ice_mass_fraction_finite
finite_values_check.py	check_ice_thickness_finite

Enumerated Value Checks Overview

- Check that a certain model output consists of valid enumerated values
- 1 out of 46 checks are of this type

Example Check

```python
def check_imelt_uses_valid_enum_values(
    test_col_ef_imelt: npt.NDArray,
) -> None:
    # Values of 2147483647 are uninitialized, and should be masked.
    masked_imelt = np.ma.masked_where(
        test_col_ef_imelt == 2147483647, test_col_ef_imelt
    )
    if NonFiniteValuesHandler.is_all_not_finite(masked_imelt):
        return CheckStatus.SKIPPED
    masked_imelt = NonFiniteValuesHandler.mask_non_finite_values(masked_imelt)

    assert np.all(masked_imelt >= 0) and np.all(masked_imelt <= 2)
```

All Enumerated Value Checks

File Name	Check Name
aggregation_and_consistency_check.py	check_imelt_uses_valid_enum_values

Relationship to Other Outputs Checks Overview

- Check that a certain model output has a certain relationship to other model outputs
- 3 out of 46 checks are of this type

Example Check

```
def check_snofrz_lyr_sums_to_snofrz_col(
    test_col_wf_qflx_snofrz_lyr: npt.NDArray,
    test_col_wf_qflx_snofrz: npt.NDArray,
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_wf_qflx_snofrz_lyr,
                                                test_col_wf_qflx_snofrz):
        return CheckStatus.SKIPPED
    test_col_wf_qflx_snofrz_lyr, test_col_wf_qflx_snofrz = (
        NonFiniteValuesHandler.mask_non_finite_values(test_col_wf_qflx_snofrz_lyr,
                                                      test_col_wf_qflx_snofrz))

    tolerance = 1E-10

    lyr_sum = test_col_wf_qflx_snofrz_lyr.sum(axis=1)
    abs_diff = np.abs(test_col_wf_qflx_snofrz - lyr_sum)

    assert np.all(abs_diff <= tolerance), "Sum does not match expected values"
```

All Relationship to Other Outputs Checks

File Name	Check Name
aggregation_and_consistency_check.py	check_snofrz_lyr_sums_to_snofrz_col
aggregation_and_consistency_check.py	check_icethick_col_is_sum
physical_bounds_check.py	check_combined_heat_content_not_less_than_soil_heat_content

Physical Laws and Restraints Checks Overview

- Check that a certain model output obeys physical laws and restraints
- 8 out of 46 checks are of this type

Example Check

```python
def check_temp_around_freezing_where_lake_is_almost_frozen(
    test_col_es_t_lake: npt.NDArray,
    test_lakestate_vars_lake_icefrac_col: npt.NDArray,
) -> None:
    if NonFiniteValuesHandler.is_all_not_finite(test_col_es_t_lake,
                                                test_lakestate_vars_lake_icefrac_col):
        return CheckStatus.SKIPPED
    test_col_es_t_lake, test_lakestate_vars_lake_icefrac_col = (
        NonFiniteValuesHandler.mask_non_finite_values(test_col_es_t_lake,
                                                      test_lakestate_vars_lake_icefrac_col))

    almost_frozen = (np.isclose(test_lakestate_vars_lake_icefrac_col, 1.0)
                     & (test_lakestate_vars_lake_icefrac_col != 1.0))

    if not np.any(almost_frozen):
        return CheckStatus.SKIPPED

    diff_to_freezing_temp = np.abs(TFRZ - test_col_es_t_lake)
    close_to_freezing_temp = diff_to_freezing_temp <= 1E-3

    assert np.all(~almost_frozen | close_to_freezing_temp), (
        "Columns are almost frozen but not close to the freezing temperature"
    )
```

All Physical Laws and Restraints Checks

File Name	Check Name
ch4_conductance_check.py	check_methane_conductance_frozen_lake
key_physical_laws_check.py	check_energy_conservation
key_physical_laws_check.py	check_freezing_latent_heat
key_physical_laws_check.py	check_melting_latent_heat
key_physical_laws_check.py	check_methane_conductance_gated_by_ice
key_physical_laws_check.py	check_methane_conductance_allowed_without_ice
physics_check.py	check_temp_around_freezing_where_lake_is_almost_frozen
physics_check.py	check_no_tke_when_surface_frozen

Workflow for Sample Generation

- ACTS generates covering arrays from 2-way to 5-way and outputs a CSV file for each, from which NetCDF files are created using a Python script (`csv2nc`).
- These are run on the model using the NetCDF Cases Sampling Plugin.

Test Suite Sizes and Timings for Cases with All Constants

For ten samples, the mean time it takes to run `elmtest` is 0.622 seconds, with a standard deviation of 0.030. This time is used to estimate how long testing each interaction level would take, assuming tests run sequentially.

*Large test cases haven't been executed yet

DOI	Case Count	Estimated Execution Time	Actual Execution Time
2	231	0 days, 00:02:24	0 days, 00:02:17
3	4,706	0 days, 00:48:47	0 days, 00:46:50
4	72,964	0 days, 12:36:24	- days, --:--:--*
5	1,009,602	7 days, 06:26:12	- days, --:--:--

ACTS Case Generation Timings for Cases with All Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.303	0.016
3	4,706	1.638	0.027
4	72,964	201.699	2.937
5	1,009,602	15421.421	171.548

`csv2nc` Script Timings for Cases with All Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.671	0.030
3	4,706	6.187	0.102
4	72,964	88.892	0.382
5	1,009,602	1155.359	16.407

Test Suite Sizes and Timings for Cases with Some Constants*

The estimated execution time for cases with some constants uses the same mean `elmtest` time as the estimated execution time for cases with all constants.

*Combinatorial cases with some constants excludes the constants originally believed to have no effect on model output.

DOI	Case Count	Estimated Execution Time	Actual Execution Time
2	231	0 days, 00:02:24	0 days, 00:02:20
3	3,685	0 days, 00:38:12	0 days, 00:37:28
4	53,170	0 days, 09:11:12	- days, --:--:--
5	688,290	4 days, 22:55:16	- days, --:--:--

ACTS Case Generation Timings for Cases with All Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.213	0.006
3	3,685	0.632	0.024
4	53,170	12.641	0.197
5	688,290	1107.923	16.651

`csv2nc` Script Timings for Cases with All Constants

DOI	Case Count	Time (seconds) Mean	Sample Standard Deviation
2	231	0.648	0.114
3	3,685	3.226	0.054
4	53,170	41.233	1.146
5	688,290	501.764	2.276

DOI 2 Results

- Degree of Interaction (DOI) 2 results in 133 test cases when only changing the values of constants known to affect the outputs of `LakeTemperature`.

33 out of 133 runs exited with an error.

Check	Passed	Skipped	Failed	Errored
check_combined_heat_content_finite	5	0	95	0
check_combined_heat_content_not_less_than_soil_heat_content	98	0	2	0
check_energy_conservation	15	67	18	0
check_energy_conservation_residual_finite	5	0	95	0
check_errsoi_almost_zero	75	0	25	0
check_freezing_latent_heat	0	100	0	0
check_ground_heat_flux_finite	100	0	0	0
check_ground_methane_conductance_finite	73	0	27	0
check_ground_net_heat_flux_finite	100	0	0	0
check_ground_sensible_heat_flux_finite	100	0	0	0
check_ice_mass_fraction_finite	100	0	0	0
check_ice_thickness_finite	33	0	67	0
check_icethick_col_is_sum	100	0	0	0
check_imelt_uses_valid_enum_values	100	0	0	0
check_lake_layer_ice_fraction_is_fraction	100	0	0	0
check_lake_temperature_finite	5	0	95	0
check_lake_transport_resistance_finite	73	0	27	0
check_lake_water_transport_resistance_not_negative	100	0	0	0
check_melting_latent_heat	0	100	0	0
check_methane_conductance_allowed_without_ice	50	50	0	0
check_methane_conductance_frozen_lake	16	84	0	0
check_methane_conductance_gated_by_ice	16	84	0	0
check_methane_conductance_non_negative	50	50	0	0
check_net_snow_melt_not_negative	100	0	0	0
check_new_snow_melt_rate_finite	100	0	0	0
check_saved_eddy_conductivity_finite	100	0	0	0
check_saved_eddy_conductivity_not_negative	100	0	0	0
check_snofrz_lyr_sums_to_snofrz_col	100	0	0	0
check_snow_depth_finite	100	0	0	0
check_snow_depth_not_negative	100	0	0	0
check_snow_freeze_rate_finite	100	0	0	0
check_snow_freeze_rate_not_negative	100	0	0	0
check_snow_layer_flag_finite	100	0	0	0
check_snow_melt_flux_not_negative	100	0	0	0
check_snow_melt_heat_flux_finite	100	0	0	0
check_snow_melt_heat_flux_not_negative	100	0	0	0
check_snow_melt_water_flux_finite	100	0	0	0
check_snow_water_equivalent_finite	100	0	0	0
check_soil_heat_content_finite	5	0	95	0
check_soil_heat_content_not_negative	93	0	7	0
check_soil_snow_temperature_finite	5	0	95	0
check_surface_absorption_finite	100	0	0	0
check_surface_absorption_fraction_is_fraction	100	0	0	0
check_temp_around_freezing_where_lake_is_almost_frozen	0	100	0	0
check_total_sensible_heat_flux_finite	100	0	0	0
check_water_snow_equivilant_not_negative	100	0	0	0

DOI 3 Results

- DOI 3 results in 3,516 test cases when only changing the values of constants known to affect the outputs of `LakeTemperature`.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

DOI 4 Results

- DOI 4 results in 50,031 test cases when only changing the values of constants known to affect the outputs of `LakeTemperature`.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

DOI 5 Data

- DOI 5 results in 689,735 test cases when only changing the values of constants known to affect the outputs of `LakeTemperature`.
- *How many combinatorial test cases generated, executed, passed, skipped, failed, crashed*

Findings from Testing

- Our test cases originally had invalid values for constants that could only be used when certain preconditions were met, which caused crashes.
- Some checks were implemented incorrectly and had to be fixed.
- For example, physics_check. check_temp_around_freezing_where_lake_is_almost_frozen originally checked to make sure the difference between the freezing temperature and a model output was close to zero but it was supposed to check that the absolute value of the difference was close to zero so that it did not ignore large negative values.
- Tests checking for things other than non-finite values had to ignore non-finite values, otherwise they'd fail and output an inaccurate error message.
- SPEL might be initializing data with NaN values that `LakeTemperature` then never overwrites or `LakeTemperature` might be outputting data with NaN values. Currently …/231 combinatorial cases with all constants have at least one model output with NaN values and …/231 combinatorial cases with some constants have at least one model output with NaN values.
- Largely, `LakeTemperature` output data was valid and within physical bounds. However, some cases resulted in some physically impossible outputs, likely due to combinations between a few model constants.